

Script SQL MySQL — SGR

Documento generado desde `script-sql.md` del repositorio

Criterio 7 de la entrega del 15 de septiembre · 10 puntos.

El archivo que se entrega es `sgr-mysql.sql`: crea la base completa de SGR en MySQL —22 tablas, 52 claves foráneas, 8 enumerados, 11 restricciones `UNIQUE`, 5 `CHECK` y 3 disparadores— y la deja poblada con datos de prueba ficticios.

Se ejecutó de verdad, contra **MySQL 8.0.46** y contra **MariaDB 10.4 y 11.4** —que es lo que trae XAMPP—, y la evidencia está en §6. No es una promesa: la rúbrica exige que corra sin errores, así que se corrió.

Sale del `DER`, que a su vez sale de `backend/prisma/schema.prisma`. Los tres artefactos dicen lo mismo, y `npm run verificar:entrega` lo comprueba columna por columna.

1. Qué pide la rúbrica §5.7, y dónde está

Lo que pide	Dónde está
Creación de la base o selección explícita del esquema	<code>sgr-mysql.sql</code> §0: <code>CREATE DATABASE sgr + USE sgr</code>
<code>CREATE TABLE</code> ordenadas por dependencia de FK	§§1 a 7 · el orden y su razón, en §3
PK y FK definidas	22 <code>PRIMARY KEY</code> y 52 <code>FOREIGN KEY</code> , con nombre del esquema
Tipos apropiados	Traducidos con la tabla de <code>der.md</code> §12.1
<code>NOT NULL</code> , <code>UNIQUE</code> , <code>DEFAULT</code> y demás restricciones	§4
Índices adicionales si el diseño los justifica	16 índices, los mismos del esquema real
<code>INSERT</code> de prueba (opcionales, recomendados)	§9 del script · §5
Se ejecuta sin errores de sintaxis ni de integridad	§6 — comprobado en tres motores, con evidencia

2. Por qué MySQL si el sistema corre en PostgreSQL

Es la **decisión D-1** del `plan`, y la consulta abierta nº 13.

La rúbrica pide el DER y el script **en MySQL**. El sistema construido corre sobre **PostgreSQL 16** y no se va a migrar: migrarlo por una entrega sería cambiar el producto para satisfacer el formato del entregable. Lo que se hace es **traducir el modelo, declarando cada equivalencia**, y decirlo en el informe.

La traducción no es automática y tiene tres puntos que conviene poder defender:

Qué cambia	PostgreSQL	MySQL	Consecuencia
Identificador	uuid nativo	CHAR(36)	Ocupa más y sus índices son más grandes. A la escala del caso no se nota
Instante	timestampz	DATETIME(3)	⚠ Se pierde la zona horaria. El equivalente fiel sería <code>TIMESTAMP</code> , pero su rango termina en 2038. La aplicación guarda todo instante en UTC y convierte al mostrar (<code>lib/fechas.ts</code>), así que la pérdida es de metadato, no de dato
Enumerado	CREATE TYPE ... AS ENUM	ENUM(...) en la columna	En MySQL el enumerado no es un tipo reutilizable: se repite en cada columna. Acá ninguno se usa dos veces, así que no duplica nada

Lo que NO cambia, y conviene decirlo porque suele suponerse lo contrario: el `UNIQUE (organization_id, periodo_id, clave)` de `parametros` se comporta **igual en los dos motores**. Ambos tratan los `NULL` como distintos entre sí, así que la unicidad de la fila por defecto (`periodo_id` nulo) no la garantiza el índice en ninguno: la exige `services/parametros.ts`. La traducción no introduce aquí una diferencia de comportamiento.

Requisito de versión: MySQL 8.0.16 o superior. Antes de esa versión las restricciones `CHECK` se analizan y **se ignoran en silencio**, y acá sostienen reglas de negocio: el formato del RUT, la coherencia de las fechas del período y el rango del ponderador. Un motor que las ignore aceptaría datos que el modelo prohíbe.

2.1 Y si lo abren en XAMPP, que no trae MySQL

XAMPP no instala MySQL: instala MariaDB. Son motores distintos desde 2009, aunque el comando siga llamándose `mysql`. Como es lo más probable que tenga a mano quien revise, el script **también se probó ahí**, y el resultado está en §6.4.

Corre igual, sin cambiarle una línea. Las tres diferencias que aparecieron son de forma, no de fondo:

	MySQL 8.0	MariaDB
Error de un <code>CHECK</code>	ERROR 3819: Check constraint 'x' is violated	ERROR 4025: CONSTRAINT 'x' failed
<code>CHECK</code> que reporta <code>information_schema</code>	5 (los nuestros)	8: los 5 nuestros más 3 que MariaDB agrega sola, porque su tipo <code>JSON</code> es un <code>LONGTEXT</code> con un <code>CHECK json_valid()</code> encima. Es más estricto, no menos
Mensaje de una FK <code>RESTRICT</code>	Muestra <code>ON DELETE RESTRICT</code>	No lo muestra, porque <code>RESTRICT</code> es su valor por defecto. En <code>information_schema</code> la regla sí figura como <code>RESTRICT</code>

Requisito de versión en MariaDB: 10.2.3 o superior, por la misma razón que en MySQL —antes de 10.2.1 los `CHECK` no se aplican— más `JSON_OBJECT()`, que llegó en 10.2.3.

3. El orden de creación, y por qué es ese

La rúbrica pide las `CREATE TABLE` ordenadas por dependencia de clave foránea. El script no desactiva `FOREIGN_KEY_CHECKS` en ningún momento: si el orden estuviera mal, no correría.

```
organizations → users → unidades_territoriales → cargos → organization_members
→ categorias_gestion → personas_usuarias → tareas → periodos → parametros
→ items_medicion → metas_item → actividades → evidencias → validaciones
→ atenciones_sociales → ausencias → ajustes → catalogo_items
→ tarea_historial → comentarios → auditoria
```

Dos cosas que lo explican:

- **organizations y users van primero porque no dependen de nadie.** `users` es la única tabla sin `organization_id`: es global, porque un mismo correo puede pertenecer a varias organizaciones con roles distintos y el rol vive en `organization_members`.
- **No hay ciclo, aunque lo parezca.** `unidades_territoriales.responsable_id` apunta a `users` y `organization_members.unidad_territorial_id` apunta a `unidades_territoriales`. Se cierra el círculo solo si `users` apuntara a alguna de las dos, y no apunta a nadie. Por eso basta con un orden lineal y no hacen falta `ALTER TABLE ... ADD CONSTRAINT` diferidos.

El verificador comprueba el orden, no la vista: recorre el script y falla si una tabla referencia a otra que todavía no se creó.

4. Las restricciones, y lo que cada una sostiene

4.1 Las 11 `UNIQUE` en 10 tablas

Cada una es una regla de negocio disfrazada de índice. La lista completa está en [der.md §12.3](#); las tres que más pesan:

- `personas_usuarias (organization_id, rut)` — una ficha por vecino en **todo el municipio**, no por delegación. Es lo que hace posible notar que la misma persona pidió lo mismo en varias delegaciones ([ADR-008](#)). Es el control que el cliente vino a buscar, y vive en un índice.
- `atenciones_sociales (actividad_id)` — es lo que hace 1:1 la relación con la actividad. Sin ese `UNIQUE` sería una 1:N más.
- `metas_item (periodo_id, item_id, funcionario_id)` — a un funcionario se le fija una sola meta por ítem y período. Cambiar de período crea filas nuevas, no edita las viejas: es lo que permite cerrar un período sin tocar su historia.

4.2 Los 5 `CHECK`

Son los mismos de la migración `20260901120000_modelo_v2_especificacion_oficial`, traducidos:

Restricción	Regla	Traducción
<code>users_rut_formato</code>	RUT canónico (ADR-001)	El operador <code>~</code> de PostgreSQL pasa a <code>REGEXP</code>

Restricción	Regla	Traducción
personas_rut_formato	Ídem	Ídem
periodo_fechas_coherentes	RN-013	Igual en los dos motores
meta_valor_positivo	RN-002	Igual
ponderador_en_rango	RN-001, en lo que un CHECK alcanza	Igual

El CHECK garantiza la **forma** del RUT. El **módulo 11** lo valida `lib/rut.ts`, porque un CHECK no puede calcular un dígito verificador.

Y hay una regla que ningún CHECK puede expresar: la RN-001 exige que los ponderadores de un funcionario en un período sumen exactamente `1.0000`. Esa suma **cruza filas**, y un CHECK solo ve la fila que se escribe. El CHECK cubre el rango `0..1`; el 100 % exacto lo exige el `PUT /metas-item` del conjunto. Está dicho en el script, donde corresponde, para que nadie lo lea como un olvido.

4.3 Los 3 disparadores

Son las dos invariantes que **no se pueden confiar a la aplicación**: una regla que solo vive en el código se salta con un script.

Disparador	Sobre	Qué impide
auditoria_sin_update	auditoria, BEFORE UPDATE	Que la propia aplicación altere la bitácora
auditoria_sin_delete	auditoria, BEFORE DELETE	Que la borre
actividad_codigo_no_cambia	actividades, BEFORE UPDATE	Que el código de evidencia cambie

`SIGNAL SQLSTATE '45000'` es el equivalente MySQL del `RAISE EXCEPTION` de PostgreSQL: aborta la sentencia con un error definido por el usuario.

Por qué el UNIQUE no bastaba para el código de evidencia: impide **repetirlo**, no impide **cambiarlo**, y RF-011 pide que sea inmutable. Son dos cosas distintas, y hacen falta las dos.

Los tres van dentro de un bloque `DELIMITER $$ ... DELIMITER ;`. `DELIMITER` es una instrucción del cliente `mysql` y de Workbench, no del servidor: quien ejecute el script desde una herramienta que no la entienda tiene que cargar esa sección aparte.

5. Los datos de prueba

La rúbrica los da por opcionales y recomendados. Acá cumplen una función concreta: **son la prueba de integridad referencial**. Recorren las 22 tablas y las 52 claves foráneas, así que si el script tuviera un error de referencias, el bloque de `INSERT` falla y se nota.

Una organización con tres delegaciones, seis personas —una por rol—, un período abierto con sus parámetros, dos cargos con sus ítems, metas que suman 100 %, un vecino, un compromiso del tubo y una actividad con su evidencia aprobada y su caso social.

Cuatro cosas están puestas a propósito, y son las que se pueden mostrar si preguntan:

Qué	Dónde	Para qué
Una delegación sin nadie con metas	«Delegación La Pampa»	Se informa como sin medición , no como 0 % (ADR-014). Es la diferencia entre «no cumplió» y «no se midió»
Un ítem inverso	«Pendientes en el tubo», menor_mejor	Superar la meta es malo . Sin esa columna el sistema premiaría tener más pendientes (ADR-009)
Quien valida no es quien registró	Elena valida lo que subió Diego	RNF-005: nadie valida lo propio
Una acción consultar en la bitácora	auditoria	No registra un cambio: registra un acceso a datos personales. Las Leyes 19.628 / 21.719 y la Ley 21.663 piden trazabilidad del acceso, no solo de la modificación (ADR-012)

⚠ **Todo es ficticio** (regla 12 del proyecto). Ningún RUT, teléfono, correo o dirección corresponde a alguien real: los RUT tienen dígito verificador válido para que pasen el CHECK, los correos son @ejemplo.demo y las IP están en el rango 192.0.2.0/24, reservado por la RFC 5737 justamente para documentación. **Las cuentas del sistema (@sgr.demo) no aparecen**, y el verificador lo comprueba.

6. Cómo se comprobó que corre

«Debe ejecutarse sin errores de sintaxis ni de integridad referencial.» — rúbrica §5.7

Se levantó un **MySQL 8.0.46** en un contenedor desechable y se ejecutó el script entero. No es una revisión a ojo. Lo mismo se hizo después con **MariaDB** (§6.4).

```
docker run -d --name sgr-mysql-prueba -e MYSQL_ROOT_PASSWORD=prueba123 -p 127.0.0.1:3306:3306 mysql:8.0
docker cp docs/entrega/sgr-mysql.sql sgr-mysql-prueba:/tmp/
docker exec sgr-mysql-prueba sh -c "mysql -uroot -pprueba123 --default-character-set=utf8mb4 <
/tmp/sgr-mysql.sql"
```

6.1 Lo que quedó creado

Salida limpia, sin un solo error ni advertencia. Y el script es **reejecutable**: se corrió dos veces seguidas sobre la misma base y la segunda dio el mismo resultado, porque empieza con DROP DATABASE IF EXISTS.

	Esperado	Creado
Tablas	22	22
Claves foráneas	52	52
Disparadores	3	3
Restricciones CHECK	5	5
Restricciones UNIQUE	11	11

6.2 Que las reglas muerden

Crear las restricciones no es lo mismo que probarlas. Se ejecutaron **trece sentencias que deben fallar**, y las trece fallaron con el error correcto:

#	Lo que se intentó	Qué pasó
1	UPDATE sobre auditoria	ERROR 1644 (45000): La auditoria es de solo insercion (RNF-008)
2	DELETE sobre auditoria	ERROR 1644 (45000) , mismo disparador
3	Cambiar actividades.codigo	ERROR 1644 (45000): El codigo de evidencia es inmutable (RF-011)
4	Anular la actividad sin tocar el código	✅ Funciona. El disparador bloquea el código, no la anulación (ADR-006)
5	RUT con puntos: 12.345.678-5	ERROR 3819: Check constraint 'personas_rut_formato' is violated
6	Período que termina antes de empezar	ERROR 3819: 'periodo_fechas_coherentes'
7	ponderador = 1.5	ERROR 3819: 'ponderador_en_rango'
8	Borrar a quien subió una evidencia	ERROR 1451 : lo detiene la primera FK RESTRICT que encuentra
9	Borrar un período con actividades	ERROR 1451: actividades_periodo_id_fkey
10	Borrar una categoría que clasifica un compromiso	ERROR 1451: tareas_categoria_id_fkey
11	Dos casos sociales para la misma actividad	ERROR 1062: atenciones_sociales_actividad_id_key — la 1:1 se sostiene
12	Repetir un RUT en la misma organización	ERROR 1062: personas_usuarias_organization_id_rut_key — ADR-008 se sostiene
13	Insertar con una FK inexistente	ERROR 1452: tareas_unidad_territorial_id_fkey

La 4 es la que importa entender: el disparador tiene que dejar pasar la anulación con motivo y bloquear solo el cambio de código. Si bloqueara todo UPDATE , rompería el flujo de RF-011 en vez de protegerlo.

6.3 La consulta de cierre

El script termina con tres consultas de comprobación. La tercera es la que muestra que el modelo hace lo que dice —**solo la evidencia aprobada suma** (RN-009)—:

```

+-----+-----+-----+-----+
---+
| funcionario      | item                                     | meta   | ponderador |
avance_validado |
+-----+-----+-----+-----+
---+
| Diego Salas Rojas | Atención de usuario, teléfono y presencial | 240.00 | 0.7000 |
1 |
| Diego Salas Rojas | Visitas domiciliarias realizadas          | 30.00  | 0.3000 |
0 |
+-----+-----+-----+-----+
---+

```

Los acentos salen bien: la base es `utf8mb4`, no `latin1`.

6.4 Y en MariaDB, lo que trae XAMPP

El mismo archivo, sin cambiarle una línea, contra **MariaDB 10.4.34** (la rama que XAMPP arrastra desde hace años) y **MariaDB 11.4.13** (la LTS actual):

	MySQL 8.0.46	MariaDB 10.4.34	MariaDB 11.4.13
Ejecución completa	✓ sin errores	✓ sin errores	✓ sin errores
Tablas	22	22	22
Claves foráneas	52	52	52
— <code>CASCADE</code> / <code>RESTRICT</code> / <code>SET NULL</code>	37 / 7 / 8	37 / 7 / 8	37 / 7 / 8
Disparadores	3	3	3
<code>UNIQUE</code>	11	11	11
Las 13 pruebas de §6.2	13 / 13	13 / 13	13 / 13
Reejecutable	✓	✓	✓
Acentos (<code>Muñoz</code> , <code>Díaz</code>)	✓	✓	✓

Las trece pruebas dan el mismo resultado en los tres motores, incluida la número 4: anular una actividad con motivo funciona, y cambiarle el código falla. Solo cambian el número de error y la redacción del mensaje (§2.1).

Los tres motores se levantaron en contenedores desechables y se eliminaron al terminar. No quedó nada instalado ni ningún puerto ocupado.

7. Cómo se comprueba la coherencia con el DER

«Toda FK representada en el DER debe existir en el script SQL. Del mismo modo, las tablas creadas en el script deben corresponder al modelo presentado en el informe.» — rúbrica

Eso se evalúa a mano. Acá se comprueba solo, **y en los dos sentidos**:

```
cd frontend
npm run verificar:entrega
```

El bloque del criterio 7 compara `sgr-mysql.sql` contra `backend/prisma/schema.prisma` y contra las migraciones:

- Las **22 tablas**, sin que falte ni sobre ninguna.
- **Columna por columna**, en los dos sentidos: ninguna columna del esquema falta en el script, y el script no inventa ninguna. Es lo que separa un script que «tiene las 22 tablas» de uno que describe el mismo modelo.
- Las **52 claves foráneas** con su `ON DELETE`, extraídas de las migraciones y no transcritas.
- Que cada FK **conserva el nombre del esquema** (`tabla_columna_fkey`). No es cosmético: es lo que deja seguir una restricción del script hasta la migración que la creó.
- Que el **orden de creación** respete las dependencias.
- Los 3 disparadores con su `SIGNAL`, los 5 `CHECK`, InnoDB y `utf8mb4`.
- Que **no aparezca ninguna cuenta** `@sgr.demo` en los datos de prueba.
- Que el script **no invente** la FK de `periodos.cerrado_por_id` ni ninguna de las tres referencias que a propósito no la tienen.

8. Lo que el script refleja y no arregla

El script describe **el modelo que el sistema tiene hoy**, no el que convendría. Cuatro cosas van tal cual, con su comentario en el propio `.sql`:

Qué	Cómo va en el script	Por qué
<code>periodos.cerrado_por_id</code>	Columna <code>CHAR(36)</code> sin clave foránea	Debería tenerla y no la tiene. Es el desvío D-d , encontrado al armar el DER (siguiente-sesion §4.bis). El script no lo corrige: corregirlo acá y no en el sistema dejaría dos modelos distintos
<code>comentarios.entidad_id</code> y <code>auditoria.entidad_id</code>	Columnas sin FK	Son polimórficas : apuntan a varias tablas según <code>entidad</code> , y ninguna base relacional declara una FK así
<code>auditoria.usuario_id</code>	Columna sin FK	Deliberado : la bitácora debe sobrevivir al borrado del usuario. <code>CASCADE</code> borraría la prueba; <code>RESTRICT</code> impediría dar de baja a nadie

Qué	Cómo va en el script	Por qué
ajustes , comentarios y tarea_historial	Tablas completas, con comentario de pendiente	Existen en el modelo y no tienen comportamiento en la aplicación (RF-025, RF-035, RF-018). Quitarlas del script haría que el script y el DER dijeran cosas distintas

9. Cómo ejecutarlo

Requiere MySQL 8.0.16+ o MariaDB 10.2.3+ (por los `CHECK`, que antes de esas versiones se ignoran en silencio).

Desde el cliente de línea de comandos:

```
mysql -u root -p --default-character-set=utf8mb4 < docs/entrega/sgr-mysql.sql
```

Desde **MySQL Workbench**: abrir el archivo y ejecutarlo entero (`Ctrl+Shift+Enter`). Workbench entiende `DELIMITER`, así que los disparadores se crean sin pasos extra.

Desde **XAMPP**: el cliente está en `xampp\mysql\bin\mysql.exe` y el motor es MariaDB, no MySQL (§2.1). Si se importa por **phpMyAdmin** en vez de por línea de comandos, conviene revisar que la sección de disparadores (§8 del script) se haya creado: el manejo de `DELIMITER` depende de la versión de phpMyAdmin, y si falla, esa sección se pega aparte en la pestaña SQL. Se comprueba con:

```
SELECT TRIGGER_NAME FROM information_schema.TRIGGERS WHERE TRIGGER_SCHEMA = 'sgr';
-- deben salir 3
```

⚠ **El script empieza con `DROP DATABASE IF EXISTS sgr`**. Es lo que lo hace reejecutable, y también lo que borra sin preguntar una base `sgr` anterior. En una base con datos, comentar esa línea primero.